Timetabl.ai: clash-free timetables for any college

Hackathon 1 | Samiya Sehgal (URN 2302662) | built with Claude Code, 21 to 26 October 2026 | code on GitHub (private repo), 96 small commits

1. The problem statement

Most colleges still build their timetables by hand at the start of every semester. Each department has to fit its sections, labs and teachers into the week, and departments compete for shared spaces like seminar halls and computer labs. It takes weeks, and the result still has clashes, uneven teaching loads and rooms that sit empty.

The task: build a web application that produces college timetables, and it must work for **any** college, not just one. Deciding what the product needs (who uses it, what data it needs, what makes one timetable better than another, what "done" means) was part of the assignment. We had to use Claude Code and commit small working steps. Judging: **core functionality** (timetables that work with no clashes), **usability** (an admin can use it without help), **design**, **advanced functionality** (harder cases) and **relevance** to real colleges.

Our class requirements on top: the clash-free timetable must come from an **algorithm, without any API key**; there must be an **agentic chatbot using the Claude API**; extra features are welcome. Our instructor suggested building it as a **SaaS product** that many colleges can use, so I did.

2. My approach and the main decisions

- **The algorithm makes the timetable, never the AI.** Google OR-Tools (CP-SAT, a constraint solver) places every class. Hard rules can never be broken; soft rules (few gaps, balanced loads, right-sized rooms) give a quality score.
- **A second, independent checker.** A separate validator, sharing no code with the solver, re-checks every timetable and every single edit. That is how I can promise "0 clashes".
- **The AI assistant is agentic but safe.** Claude uses tools to look things up and to *prepare* changes (moves, makeup classes, bookings). It never changes anything by itself: the person sees a card and clicks Apply. It works with the logged-in person's own permissions, so a student's assistant can only read.
- **SaaS from the start.** Any college signs up and sets itself up: 1 or 2 shifts, periods, lab length, a flexible lunch window, holidays and rules are all data, not code. Colleges are kept fully separate.
- **Core first.** Setup, generate, view and change safely were built and polished before anything else. Every change goes through repair, check, preview, apply, activity log, notifications, and can be undone.

3. How I built it (my journey)

Planning first. I did several rounds of questions with Claude Code before writing code: roles, what each role may do, the algorithm, what goes in version 1 and what waits. All of it went into one project guide (CLAUDE.md) that every later step followed.

Stage	What it delivered
1. Engine	Solver, independent validator, quality score, plain-language errors for impossible data, demo college
2. See it	Web app with section, teacher and room views; Generate button with live progress
3. Edit it	Drag and drop with green / amber / red slots, minimal repair, preview, one-day changes, undo
4. Real SaaS	Sign-up, roles, invites, student logins, college isolation, cloud database (Supabase Postgres)
5. Set it up	Setup wizard, Excel upload with row errors, fair teacher suggestions, Ready flow, electives, rules
6. Chatbot	Claude assistant with role-limited tools; changes come back as cards you Apply
7. Ship	Publish, Excel/PDF export, notifications, teacher change requests, attendance, Today view, makeup finder
8. Version 2	Hinglish + voice, teacher booking, auto-sectioning, Super Admin platform, mid-semester regenerate

Moving to version 2. Before adding anything new I audited version 1 as each role. That found real problems, which I fixed: teacher suggestions were unfair (one teacher 11 periods, another 0), the sample-college button could overwrite real data, and a damaged Excel file crashed the server. Then I built the version 2 features listed above. **Last, a usability pass:** I counted the clicks for every daily task per role and rebuilt Home and navigation around them. A teacher absence went from **19 clicks to 3**; most core tasks now take 3 clicks or fewer.

Who uses it. Each person sees only their own part, and the assistant follows the same limits.

Role	What they do	What they see
College Admin	Sets up the college, invites people, generates and publishes, changes anything, undoes changes	Everything in their college
Department Coordinator	Fills the department's data, accepts teacher suggestions, marks Ready, approves teachers' requests	Own department
Teacher	Today's classes, attendance, one-day changes to own lectures, asks for permanent moves, office hours	Only own lectures
Student	Own week and today, attendance, doubt-slot booking; the assistant can only read for them	Only own classes
Platform owner	Runs the SaaS: every college's progress and health, plans, pause. Never edits college data	All colleges, no college data

4. What is ready, and which parts are agentic

App + Agentic = it works in the app, and the assistant can also do it: it uses tools and prepares the change, a person clicks Apply. **App + Answers** = the assistant can look it up and reply, it changes nothing. **No AI** = only the algorithm or the app does it (on purpose for the timetable itself).

Feature	What it does	Who	AI?
Generate timetable	Whole college, labs as blocks with parallel batches, combined classes, electives across departments, 2 shifts, lunch inside the mess window for every section every day	Admin	No AI
Independent checker	Re-checks every timetable and edit; shown on every view as "0 clashes"	All	No AI
Mid-semester regenerate	After a data change only the classes that must move do	Admin	No AI
Move a class	Drag and drop with live slot colours, minimal repair, preview, undo	Admin, Coordinator	App + Agentic
One-day change, makeup class	Cancel or move for one date; finds a time when teacher, students and room are free	Staff, Teacher	App + Agentic
Teacher away	Cancel all of a teacher's classes for given days in one step, everyone told	Admin, Coordinator	No AI (yet)
Home / Today, Changes page	Who is away, what is not happening, latest changes with Undo; requests to approve	All	No AI
Setup wizard and checklist	Shifts, periods, lunch window, holidays; 7-step checklist with links to fix	Admin	No AI
Excel import / export	Upload per department with row errors (sheet comes back marked); Excel/PDF downloads	Staff	No AI
Auto-sectioning, teacher suggestions	Whole-year list into balanced sections; fair teacher allotment	Staff	No AI
Attendance	Teachers mark; students see % against 75% and how many they can miss	All	App + Answers
Teacher booking	Office hours, doubt slots, meetings at a time all chosen teachers are free	All	App + Agentic
Free rooms, where is a teacher, free time, why here	Quick answers from the live timetable	All	App + Answers
Language and voice	English gets English, Roman Hinglish gets Hinglish; speak and hear the answer	All	Assistant itself
Super Admin platform	Every college's progress, usage and health; plans, read-only, pause	Platform owner	App + Agentic

5. Use cases: a normal week, by role

- **College Admin:** signs up, answers the setup wizard, follows the checklist, clicks Generate (about 45 seconds), checks it, publishes. Every morning Home shows today: if a teacher is away, three clicks cancel their classes and tell everyone, or the assistant is asked to sort it out.
- **Department Coordinator:** downloads the department's Excel sheet, fills it, uploads it (bad rows come back marked), accepts the fair teacher suggestions, marks the department Ready, approves teachers' move requests.
- **Teacher:** opens Home to see what is next and where, takes attendance in one tap per absent student, cancels or moves a lecture for a day, asks the Coordinator to move one for good, sets office hours.
- **Student:** sees "next class in 20 minutes, LH-102", the week with any changes, attendance against 75%, books a doubt slot, and can simply ask: *"kal meri pehli class kab hai?"* and gets the answer in Hinglish.

6. Results and testing

Demo college (12 sections, 40 teachers, 15 rooms, 2 shifts)	238 classes placed, **0 clashes**, quality 86/100, about 45 s
Mid-semester change (one teacher blocks a time)	**1 of 238** classes moved in about 4 s; a fresh run would keep only 17 in place
Automated tests	**147 passing** (107 engine/API, 40 assistant), incl. role and college isolation
Live check today, all 5 roles	**40 of 40** checks passed; 24 pages opened with no errors in the browser
Assistant, live	Answers from real data in English and Hinglish; refuses when a student asks it to change something

7. What is not done yet, and why

- **Phase 2, the assistant handling a teacher's leave end to end:** rank substitutes fairly, try every plan on a copy, then one plan card to approve. Being built next; today it can cancel and find makeup times.
- **Phase 3 and 4:** upload a college's existing timetable and compare it with ours; "what if" questions and a "Why here?" button on each class.
- **Online deployment:** ready (deploy guide written) but my free hosting trial ran out; it runs on the cloud database from my laptop for now.
- **Kept for later on purpose:** WhatsApp/SMS and email alerts, calendar sync, QR posters, substitute log, TV board.

8. Built with

Next.js (web app, light and dark, works on phones) | FastAPI in Python (backend) | Google OR-Tools CP-SAT (timetable algorithm) | Supabase Postgres (database) | Claude API, Claude Opus 5.5 (assistant only) | Claude Code for planning, writing, testing and debugging, with every step committed to Git.